

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики
(повне найменування назва факультету)
Кафедра дискретного аналізу та інтелектуальних систем
(повна назва кафедри)

КУРСОВА РОБОТА

Прогнозування ефективності логістичних операцій за
допомогою машинного навчання

Студента 3 курсу, групи ПМІ-33
напряму підготовки 122 – комп'ютерні науки
Лук'янчук Д.Є.
(прізвище та ініціали)

Керівник доц. Баранов М.В.
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____ Оцінка: ECTS _____

Зміст

ВСТУП	2
1 ТЕОРЕТИЧНІ ТА МАТЕМАТИЧНІ ОСНОВИ	4
1.1 Проблематика прогнозування в логістиці	4
1.2 Постановка задачі як задачі бінарної класифікації	4
1.3 Порівняльний аналіз методів машинного навчання	5
1.4 Математичний принцип роботи XGBoost	6
1.5 Огляд суміжних досліджень	7
1.6 Система метрик для об'єктивної оцінки	7
2 ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АЛГОРИТМІЧНЕ ЗАБЕЗПЕЧЕННЯ	9
2.1 Архітектура системи	9
2.2 Вхідні дані та виявлена проблема	10
2.3 Алгоритм нормалізації назв країн	10
2.4 Геокодування та розрахунок відстані	11
2.5 Гібридна модель погоди	12
2.6 Формула реалістичної прибутковості	13
2.7 Інженерія ознак	13
3 ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ	15
3.1 Набір даних та поділ вибірки	15
3.2 Крос-валідація	15
3.3 Оптимізація гіперпараметрів через Optuna	16
3.4 Результати фінальної моделі	16
3.5 Аналіз ROC та Precision-Recall	18
3.6 Оптимізація порогу класифікації	20
3.7 Інтерпретація через метод SHAP	21
3.8 Приклад роботи системи	23
ВИСНОВКИ	25
Список використаних джерел	27

ВСТУП

Актуальність теми. Сьогодні логістика — це основа світової економіки. Але чим далі, тим важче в ній працювати: ланцюги постачання стають складнішими, залучається більше транспорту й посередників, а правила в різних країнах постійно змінюються. Зараз недостатньо лише доставити вантаж — потрібно наперед розуміти, чи принесе ця операція прибуток. Проблема в тому, що точно спрогнозувати заробіток дуже важко: дані бувають неповними, а зовнішнє середовище непередбачуваним.

Машинне навчання дає потужний інструмент для роботи з такими даними. Сучасні алгоритми, зокрема градієнтний бустинг, здатні виявляти зв'язки між сотнями параметрів, які людина вручну просто не помітить. Система, побудована на таких алгоритмах, дає змогу компаніям не лише виправляти помилки постфактум, а й передбачати ризики ще до підписання контракту.

Метою роботи є розробка та дослідження системи класифікації логістичних операцій на прибуткові і збиткові з використанням алгоритму XGBoost та інтеграції зовнішніх даних — погодних, паливних і макроекономічних характеристик країн призначення.

Завдання дослідження:

1. Проаналізувати підходи до прогнозування в логістиці та порівняти методи машинного навчання для табличних даних;
2. Сформулювати задачу оцінки прибутковості як задачу бінарної класифікації;
3. Розробити модульну архітектуру системи з інтеграцією зовнішніх та локальних довідкових даних (геокодування, погода, макроекономіка);
4. Реалізувати алгоритм нормалізації текстових географічних даних для усунення мовних розбіжностей у наборі даних;
5. Спроектувати формулу розрахунку реалістичної прибутковості з динамічними штрафами;
6. Провести навчання та оптимізацію моделі XGBoost і дати технічну інтерпретацію результатів.

Об’єктом дослідження є процеси автоматизованої оцінки фінансової ефективності логістичних операцій.

Предметом дослідження є методи градієнтного бустингу, алгоритми інженерії ознак та способи інтеграції зовнішніх даних у ML-систему.

Практичне значення. Розроблена система може бути основою для модуля підтримки прийняття рішень у логістичних компаніях — вона дозволяє в отримати прогноз і побачити, за рахунок чого операція може бути збитковою.

РОЗДІЛ 1. ТЕОРЕТИЧНІ ТА МАТЕМАТИЧНІ ОСНОВИ

1.1 Проблематика прогнозування в логістиці

Логістика — одна з найбільш насичених даними галузей, де прибутковість кожного окремого замовлення залежить від десятків факторів одночасно. Класичний підхід — будувати тарифну сітку на основі відстані і ваги товару — давно вже не відображає реальності. Ціна дизелю може стрибнути за тиждень на 20%, непередбачена злива затримує авіавантаж на добу, а митна ставка змінюється залежно від поточного LPI країни призначення [15].

У науковій літературі застосування машинного навчання в логістиці охоплює широкий спектр задач: від прогнозування затримок доставки до оптимізації витрат та прибутку. У роботах [6, 7] показано, що ансамблеві та бустингові підходи є перспективними для аналізу реальних логістичних даних, особливо коли йдеться про складні нелінійні залежності та велику кількість факторів. Подібні результати щодо високої ефективності XGBoost наведено і в [8], де цей алгоритм використовувався для аналізу даних ланцюгів постачання та прогнозування їхньої операційної ефективності.

1.2 Постановка задачі як задачі бінарної класифікації

З точки зору комп'ютерних наук, розглянута задача — це бінарна класифікація. Ця задача розв'язувана методом навчання з вчителем. Ми маємо набір прикладів $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, де $\mathbf{x}_i$ — вектор ознак i -го замовлення, а $y_i \in \{0, 1\}$ — мітка класу:

- $y = 1$ (Прибутково) — дохід від замовлення перевищує всі реальні витрати (паливо, погодні ризики, митниця);
- $y = 0$ (Збитково) — замовлення не покриває операційних витрат.

Важливо, що мітка y не збігається з первинними показниками фінансового результату із набору даних. Вона розраховується за формулою з динамічними штрафами, яку описано в підрозділі 2.6. Це дозволяє сформувати релевантнішу цільову змінну, ніж звичайний індикатор бухгалтерського прибутку.

Модель навчається знаходити функцію $f : \mathbb{R}^M \rightarrow [0, 1]$, яка для нового замовлення $\mathbf{x}$ повертає *ймовірність* того, що воно прибуткове. Фінальне рішення приймається за порогом τ :

$$\hat{y} = \begin{cases} 1, & f(\mathbf{x}) \geq \tau \\ 0, & f(\mathbf{x}) < \tau \end{cases} \quad (1)$$

Вибір τ — окремий і важливий крок, який буде детально розглянуто у розділі 3.

1.3 Порівняльний аналіз методів машинного навчання

Для вирішення задачі бінарної класифікації на структурованих табличних даних існує кілька підходів. У межах теоретичного аналізу було розглянуто три основні методи:

Логістична регресія [4]. Найпростіший варіант, у якому модель шукає лінійну межу між класами у просторі ознак. Вона характеризується високою швидкістю обчислень та чіткою інтерпретованістю, оскільки аналіз вагових коефіцієнтів дозволяє прямо оцінити ступінь впливу кожного параметра на фінальний результат. Проте лінійна модель не здатна вловлювати складні нелінійні взаємодії між ознаками — наприклад, коли прибутковість логістичної операції різко падає саме за умови поєднання великої відстані маршруту та несприятливих погодних умов.

Випадковий ліс (Random Forest) [4]. Ансамблевий метод, який будує велику кількість незалежних дерев рішень на випадкових підмножинах даних та ознак, після чого усереднює їхні відповіді. Цей підхід ефективно справляється з нелінійними залежностями та є стійким до перенавчання. Головними недоліками методу є значне уповільнення процесу навчання на великих масивах даних, а також потреба в обов'язковому попередньому кодуванні великої кількості категоріальних ознак.

Градiєнтний бустинг, зокрема XGBoost [1]. На відміну від паралельного навчання у Random Forest, тут дерева рішень будуються послідовно, де кожна наступна модель мінімізує помилки попередніх. За результатами масштабних порівняльних досліджень на стандартних наборах даних [9], цей підхід демонструє найвищу ефективність та точність саме

на структурованих таблицях. Додатковою перевагою XGBoost є вбудована оптимізована підтримка категоріальних ознак, що є критично важливим для аналізу географічних назв та типів доставки у поточному дослідженні.

Аналіз суміжних наукових робіт та теоретичних особливостей алгоритмів показує, що для структурованих логістичних даних із числовими та категоріальними ознаками алгоритми градієнтного бустингу є найбільш доцільним вибором. Саме тому в межах цієї роботи основним інструментом реалізації та дослідження було обрано алгоритм XGBoost.

1.4 Математичний принцип роботи XGBoost

XGBoost [1] будує ансамбль із K дерев рішень. Прогноз для i -го прикладу — це сума виходів всіх дерев:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i) \quad (2)$$

На кожному кроці t алгоритм додає нове дерево f_t , яке мінімізує цільову функцію:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (3)$$

де ℓ — функція втрат (для бінарної класифікації це логістична втрата), $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ — член регуляризації, що штрафує за складність дерева (T — кількість листків, w — їхні ваги).

Щоб ефективно мінімізувати (3), алгоритм використовує апроксимацію другого порядку Тейлора. Для кожного прикладу обчислюються:

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2} \quad (4)$$

Оптимальна вага для листка j тоді рахується аналітично:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (5)$$

де I_j — множина прикладів, що потрапили в листок j . У цьому полягає

ключова перевага алгоритму XGBoost: замість повного градієнтного спуску знаходиться аналітичний оптимум для кожного листка за одну ітерацію.

1.5 Огляд суміжних досліджень

У роботі [6] автори досліджували можливості алгоритму XGBoost для класифікації затримок замовлень у сфері e-commerce на наборі даних з понад 96 тис. записів. Модель досягла загальної точності 93.24%, однак автори вказують на суттєву проблему дисбалансу класів: recall для затриманих замовлень виявився близьким до нуля, що є типовою складністю задач бінарної класифікації при нерівномірному розподілі міток. Дослідження [7] застосовує методи машинного навчання (XGBoost та рекурентні нейронні мережі) для прогнозування попиту та управління ризиками в ланцюгах постачання, вводячи комплексні метрики оцінки CAE та CAE-ESG, що виходять за межі стандартної точності.

Окрему увагу варто приділити роботі [8], де оптимізований XGBoost застосовується для прогнозування фінансової та операційної ефективності (прибутковості) ланцюгів постачання в контексті Industry 4.0. Проте більшість існуючих публікацій обмежується внутрішніми даними логістичних систем, не враховуючи динаміку зовнішнього середовища — погоди, паливних цін та макроекономічних індикаторів.

1.6 Система метрик для об'єктивної оцінки

Для стійкості до дисбалансу класів використання стандартної метрики точності дає хибне відчуття якості моделі. Наприклад, якщо базова модель завжди прогнозуватиме найбільш розповсюджений клас — «прибутково», вона автоматично демонструватиме високий загальний показник точності, але при цьому не виявить жодного збиткового замовлення. Оскільки саме збиткові операції несуть головний фінансовий ризик для логістичної компанії, виникає необхідність у застосуванні альтернативних метрик оцінки.

Тому було використано набір метрик, які стійкі до дисбалансу [4]:

- **ROC AUC** — показує, наскільки добре модель відрізняє прибуткові замовлення від збиткових незалежно від порогу класифікації. Значе-

ння 0.5 — це випадкове вгадування, 1.0 — ідеальна модель.

- **Macro F1-score** — узагальнююча метрика, що базується на показниках точності та повноти:

- **Precision (Точність)** визначає частку правильних прогнозів серед усіх об'єктів, віднесених алгоритмом до певного класу:

$$Precision = \frac{TP}{TP+FP}.$$

- **Recall (Повнота)** відображає здатність моделі ідентифікувати всі істинні об'єкти цього класу: $Recall = \frac{TP}{TP+FN}.$

Macro F1-score обчислюється як середнє гармонійне між цими двома показниками окремо для кожного класу. Це критично важливо, оскільки нас однаково цікавить якість виявлення і прибуткових, і збиткових рейсів.

- **MCC (Коефіцієнт Меттьюза)** — вважається однією з найбільш об'єктивних метрик для бінарної класифікації при дисбалансі [5]. Він враховує всі чотири типи помилок одночасно:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

де TP , TN , FP , FN — істинно/хибно позитивні та негативні результати.

- **Brier Score** — середньоквадратична похибка між передбаченою ймовірністю та реальною міткою. Він оцінює не тільки правильність класу, а й якість (калібрування) самих ймовірностей. Чим менший показник — тим краще.
- **Average Precision (AP)** — площа під кривою Precision-Recall. Особливо важлива при дисбалансі, бо надзвичайно чутлива до якості виявлення рідкіснішого класу (у розглянутому випадку — збиткових замовлень).

РОЗДІЛ 2. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АЛГОРИТМІЧНЕ ЗАБЕЗПЕЧЕННЯ

2.1 Архітектура системи

Головна особливість та науково-практична новизна запропонованого підходу полягає у комплексному збагаченні базового набору даних (Data Enrichment). На відміну від існуючих рішень, архітектура розробленої системи передбачає інтеграцію зовнішніх факторів: погодних ознак, цін на нафту та макроекономічних індикаторів (ВВП та індекс LPI країн призначення). Крім того, важливим авторським внеском є впровадження спеціалізованого етапу попередньої обробки для виправлення системних помилок кодування іспанських назв міст у наборі даних DataCo, що дозволило значно покращити якість просторових ознак моделі.

При проектуванні самої програми було обрано модульну архітектуру з принципом *Separation of Concerns*: кожен модуль відповідає за одну конкретну задачу і нічого більше. Проєкт написаний на Python 3.11 і складається з семи модулів¹.

Таблиця 1 – Структура програмного комплексу

Модуль	Що робить
<code>config.py</code>	Всі константи, словники нормалізації назв країн, макроекономічні дані, митні ставки
<code>geo_service.py</code>	Геокодування через Nominatim API, розрахунок відстані за формулою гаверсинусів, кешування результатів
<code>weather_service.py</code>	Отримання погоди з Open-Meteo API або офлайн-апроксимація
<code>economics_service.py</code>	GDP PPP, LPI, ціни Brent, розрахунок митного збору
<code>pipeline.py</code>	Головний ETL-конвеєр: від сирих CSV до готових ознак
<code>train_model.py</code>	Навчання XGBoost, оптимізація через Optuna, SHAP, метрики
<code>main.py</code>	CLI-інтерфейс: запуск пайплайну, навчання, ручне передбачення

¹Повний вихідний код системи доступний у публічному репозиторії: <https://github.com/Delukich/Coursework.git>

Система підтримує спеціальний офлайн-режим за допомогою змінної середовища `LOGISTICS_OFFLINE=1`. У цьому режимі замість прямих API-запитів використовуються локальні кеші, багаторівневі fallback-механізми та математичні апроксимації, що дозволяє формувати заповнений вектор ознак навіть за відсутності доступу до зовнішніх сервісів.

2.2 Вхідні дані та виявлена проблема

Для навчання моделі було використано набір даних **DataCo Smart Supply Chain** [14] — 180 519 записів про реальні логістичні операції компанії за 2015–2018 роки, 53 вхідні колонки. Такий значний обсяг даних дозволив успішно навчити алгоритм XGBoost та виявити стійкі закономірності логістичних процесів.

При первинному аналізі було виявлено нетривіальну проблему: усі назви країн у наборі даних записані іспанською мовою. Наприклад: **Francia**, **Alemania**, **Países Bajos**. Це критично, бо використаний модуль економіки шукає дані за англійською назвою. Через цю мовну розбіжність значна частина замовлень помилково отримувала за замовчуванням значення $GDP = 15\,000$ замість реального показника відповідної ринкової зони. Наприклад, для **Alemania** (Німеччина) фіксувалося базове значення $GDP = 15\,000$ замість фактичного 52 000, що створювало суттєве викривлення вхідних параметрів.

2.3 Алгоритм нормалізації назв країн

Для розв’язання цієї проблеми у файлі `config.py` було реалізовано функцію `normalize_country_name()`. Вона працює у чотири кроки:

1. **Прямий переклад** через словник `SPANISH_TO_ENGLISH` (понад 90 записів). Це найшвидший крок — $O(1)$ lookup. Покриває найчастіші випадки.
2. **Фіксовані скорочення** через `QUICK_FIX` (наприклад, «usa» → «USA», «uk» → «UK»).
3. **Нечіткий пошук** через бібліотеку `ruscountry` — стандартизована база країн за ISO 3166.

4. **Алгоритм Левенштейна** через `fuzzywuzzy` з порогом схожості 92% для основного пошуку та 96% для fallback-випадків. Це допомагає обробити помилки в написанні.

У результаті застосування розробленого алгоритму частку замовлень, для яких використовувалися макроекономічні показники за замовчуванням, вдалося мінімізувати до критично низького рівня. Для переважної більшості країн це дозволило успішно скоригувати вхідні ознаки моделі й забезпечити прив'язку до реального економічного контексту:

Таблиця 2 – Приклади виправлення GDP після нормалізації

У наборі даних	Після нормалізації	GDP до	GDP після
Francia	France	15 000	45 300
Alemania	Germany	15 000	52 000
Reino Unido	UK	15 000	43 000
Países Bajos	Netherlands	15 000	61 000
Japón	Japan	15 000	48 000

2.4 Геокодування та розрахунок відстані

Для обчислення відстані від складу до пункту призначення необхідно визначити координати міста доставки. Цей процес геокодування було реалізовано через Nominatim API від OpenStreetMap [13].

З метою забезпечення високої швидкості обробки даних та оптимізації взаємодії з мережевими сервісами при опрацюванні понад 180 тис. замовлень, у системі було реалізовано дворівневий механізм кешування:

1. **Декоратор** `@lru_cache(maxsize=20000)` — забезпечує миттєвий доступ до результатів у RAM під час роботи скрипта.
2. **Персистентний файл** `geocode_cache.json` — зберігає результати між запусками програми. Запис у файл реалізовано атомарно (через тимчасовий файл та `os.replace`), щоб уникнути пошкодження даних при раптових збоях.

Завдяки цьому при повторному запуску пайплайну геокодування кількох тисяч унікальних пар «місто–країна» виконується значно швидше, ніж

при повному завантаженні «з нуля», оскільки більшість результатів пере-використовується з кешу.

Для оцінки відстані використовується багаторівневе визначення координат. Спочатку система намагається отримати координати міста, а в разі недоступності точного геокодування — застосовує координати на рівні країни, регіону або ринку. Такий підхід дозволив практично повністю усунути використання жорстких дефолтних значень: після застосування нового пайплайна частка записів з фіксованою відстанню 5000 км була знижена до мінімуму.

Відстань між складом і містом доставки розраховується за формулою гаверсинусів [10], яка враховує сферичність Землі. Для трансконтинентальних маршрутів це критично важливо, оскільки звичайна евклідова метрика давала б похибку:

$$a = \sin^2 \left(\frac{\Delta\varphi}{2} \right) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2 \left(\frac{\Delta\lambda}{2} \right) \quad (6)$$

$$d = 2 \cdot R \cdot \text{atan2} \left(\sqrt{a}, \sqrt{1-a} \right), \quad R = 6371 \text{ км} \quad (7)$$

2.5 Гібридна модель погоди

Архітектура системи підтримує використання Open-Meteo API [12], який може повертати архівні значення температури та опадів за вказаними координатами і датою. Однак у відтворюваному офлайн-середовищі, використаному для фінального експерименту, погодні ознаки формувалися переважно за допомогою математичної апроксимації та fallback-механізмів.

Метод `_estimate_climate()` апроксимує температуру через тригонометричну модель сезонних циклів:

$$T(\varphi, M) = T_{\text{mean}}(\varphi) + A(\varphi) \cdot \cos \left(\frac{2\pi(M - M_{\text{peak}})}{12} \right) \quad (8)$$

де $T_{\text{mean}} = \max(-2, 28 - 0.42 |\varphi|)$, $A = \min(18, 2 + 0.22 |\varphi|)$, $M_{\text{peak}} = 7$ для Північної і $M_{\text{peak}} = 1$ для Південної півкулі (M — номер місяця).

Аналогічна логіка з адаптованими коефіцієнтами залежно від кліматичного поясу ($|\varphi| < 15^\circ$, $15^\circ\text{--}30^\circ$, $> 30^\circ$) використовується для оцінки опадів. Завдяки цьому система формує адекватні наближені оцінки пого-

ди навіть за відсутності прямого мережевого доступу до історичних даних. Впровадження цієї гібридної моделі дало змогу кардинально покращити якість погодних ознак: масовий дефолт температури (15.0 °C) зник, а частка записів з нульовими опадами як значення за замовчуванням зменшилася до мінімуму.

2.6 Формула реалістичної прибутковості

Поле «Profit» не бралось прямо з набору даних, бо це поле не враховує реальні логістичні витрати. Натомість розроблено формулу, яка перераховує прибуток із динамічними штрафами:

$$P_{\text{real}} = P_{\text{base}} - C_{\text{dist}} - C_{\text{weather}} - C_{\text{tariff}} \quad (9)$$

Кожна компонента:

$$C_{\text{dist}} = \frac{d}{1000} \cdot \frac{\text{fuel}}{60} \cdot k_{\text{ship}} \quad (10)$$

$$C_{\text{weather}} = \begin{cases} 2.0, & \text{rain} > 5 \text{ мм} \\ 0, & \text{інакше} \end{cases} \quad (11)$$

$$C_{\text{tariff}} = \text{total} \cdot r_{\text{cat}} \cdot \left(1 + \max\left(0, \frac{3 - \text{LPI}}{50}\right) \right) \quad (12)$$

де $k_{\text{ship}} \in \{1.0, 1.2, 2.0, 3.0\}$ — коефіцієнт режиму доставки (Standard / Second / First / Same Day), r_{cat} — базова митна ставка для категорії товару, а LPI-корекція збільшує тариф для країн з поганою логістичною інфраструктурою. Якщо $P_{\text{real}} \geq 0$, то `Is_Profitable` = 1, інакше 0.

2.7 Інженерія ознак

Окрім «сирих» полів набору даних, сконструйовано низку нових ознак. Головна ідея: *модель повинна отримати вже «переварену» інформацію, а не рахувати взаємодії між ознаками самостійно*. Це прискорює навчання і підвищує точність.

Варто окремо зупинитися на `ship_sla_rank`. Початково в системі була ознака `shipping_speed_ratio = sched_days / (dist / 500 + 1)`, що ділила заплановані дні доставки на «теоретичну тривалість за 500 км/день».

Але при аналізі виявилось, що поле **Days for shipment (scheduled)** має лише чотири унікальних значення (0, 1, 2, 4) і 100% відповідає типу доставки: це просто **код SLA-класу**, а не реальна кількість днів. Ділити код класу на відстань — не має змісту, і більшість рядків отримували значення менше 1.0, що зробило ознаку майже константою. Її замінено на **ship_sla_rank**, і це покращило якість моделі.

Усі розрахунки реалізовані через `df.map()` і векторизацію NumPy [11] — без циклів `for`. Це важливо для продуктивності: обробка 180 тис. рядків займає хвилини, а не години.

Таблиця 3 – Нові інженерні ознаки системи

Ознака	Що означає	Тип
<code>price_per_km</code>	Відношення суми замовлення до відстані. Ключовий індикатор маржинальності маршруту	Числова
<code>fuel_x_distance</code>	Добуток ціни нафти на відстань / 1000. Оцінка паливного навантаження	Числова
<code>ship_sla_rank</code>	Клас SLA: Same Day=0, First=1, Second=2, Standard=3. Замінив невалідну ознаку <code>shipping_speed_ratio</code>	Числова
<code>ship_cost_per_km</code>	Коефіцієнт режиму × паливо / відстань	Числова
<code>discount_x_qty</code>	Знижка × кількість одиниць	Числова
<code>gdp_x_lpi</code>	GDP PPP × LPI країни. Показує «якість» ринку	Числова
<code>inflation_risk</code>	1, якщо інфляція країни > 5%	Бінарна
<code>is_cold</code>	1, якщо температура < 0°C	Бінарна
<code>is_heavy_rain</code>	1, якщо опади > 10 мм	Бінарна
<code>dest_lat_abs</code>	широта пункту призначення	Числова
<code>cross_hemisphere</code>	1, якщо склад і ціль у різних півкулях	Бінарна
<code>order_quarter</code>	Квартал замовлення (1–4)	Числова

РОЗДІЛ 3. ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

3.1 Набір даних та поділ вибірки

Після проходження через пайплайн (`pipeline.py`) отримано набір даних із **180 519 рядків** і **38 ознак** та цільову змінну `Is_Profitable`. Баланс класів: 65.05% прибуткових (клас 1) і 34.95% збиткових (клас 0).

Критично важливим кроком є правильний поділ даних. У роботі використано **тришаровий поділ**, щоб уникнути витoku даних — ситуації, коли тестові дані якимось чином впливають на процес навчання:

- **Train (70%, 126 358 записів)** — на цих даних навчається модель, і тільки на них розраховуються медіани для заповнення пропусків.
- **Validation (15%, 27 083 записи)** — ця вибірка використовується за допомогою Optuna для добору гіперпараметрів і підбору оптимального порогу класифікації τ .
- **Test (15%, 27 078 записів)** — відкладається від самого початку і використовується для підсумкової оцінки якості моделі.

Фінальна модель перенавчається на **Dev set = Train + Validation**, щоб максимально використати дані. Поділ виконано стратифіковано, тобто в кожній частині зберігається той самий баланс класів, що і в усьому наборі даних.

3.2 Крос-валідація

Перед оптимізацією гіперпараметрів було проведено **5-фолдову крос-валідацію** на Dev set для оцінки стабільності базової моделі. Результати наведено в таблиці 4.

Мале стандартне відхилення ($\text{std} = 0.0025$ для AUC) означає, що модель стабільна: вона не «заточена» під якусь конкретну підвибірку, а навчилася на загальних закономірностях.

Таблиця 4 – Результати StratifiedKFold CV (5 фолдів)

Fold	AUC	Accuracy	Macro F1
1	0.6560	0.6949	0.5841
2	0.6620	0.6961	0.5876
3	0.6546	0.6922	0.5807
4	0.6578	0.6968	0.5880
5	0.6563	0.6960	0.5849
Mean $\pm$ std	0.6573 $\pm$ 0.0025	0.6952 $\pm$ 0.0016	0.5850 $\pm$ 0.0026

3.3 Оптимізація гіперпараметрів через Optuna

Замість ручного перебору параметрів було використано фреймворк Optuna [3]. Він реалізує алгоритм TPE (Tree-structured Parzen Estimator) — байєсівський підхід, де кожна нова спроба будується з урахуванням результатів попередніх, а не навмання. Це набагато ефективніше за Grid Search: за 50 ітерацій Optuna знаходить конфігурацію, яку Grid Search знайшов би за тисячі перевірок.

Optuna шукала параметри в такому просторі: `n_estimators`: 100–600, `max_depth`: 3–10, `learning_rate`: 0.005–0.2 (логарифмічна шкала), `subsample`: 0.5–1.0, `colsample_bytree`: 0.5–1.0, `min_child_weight`: 1–15, `gamma`: 0–2, `reg_alpha` (L1): 0–1, `reg_lambda` (L2): 0.5–3.

Найкращий результат було отримано в trial #49, де validation ROC AUC досягло значення 0.6716. Оцінка проводилася саме на Validation, тоді як Test залишався відкладеним до етапу підсумкового оцінювання моделі.

3.4 Результати фінальної моделі

Після того як Optuna знайшла найкращу конфігурацію гіперпараметрів, модель було перенавчено на повному Dev set (Train + Validation) і далі оцінено на відкладеній тестовій вибірці. Для коректного порівняння також було використано naïve prevalence baseline: кожному замовленню призначається однакова ймовірність прибутковості, що дорівнює частці прибуткових замовлень на тестовій вибірці ($p = 0.6505$). Оскільки це значення перевищує стандартний поріг 0.5, такий baseline фактично класифікує всі замовлення як прибуткові. Результати фінального оцінювання наведено в таблиці 5.

Таблиця 5 – Порівняння фінальної моделі з naive prevalence baseline на тестовій вибірці

Метрика	Запропонована модель	Naive baseline
Accuracy	0.6746	0.6505
Balanced Accuracy	0.6177	0.5000
ROC AUC	0.6737	0.5000
Average Precision	0.7630	0.6505
MCC	0.2513	0.0000
Cohen Kappa	0.2476	0.0000
Brier Score	0.1999	0.2274
Macro F1	0.6214	0.3941

Naive baseline: кожному замовленню призначається однакова ймовірність прибутковості, рівна частці прибуткових замовлень на тестовій вибірці ($p = 0.6505$). Після застосування стандартного порогу 0.5 така схема класифікує всі замовлення як прибуткові.

Як видно з таблиці 5, запропонована модель перевершує naive baseline за всіма ключовими критеріями. Це означає, що модель не просто відтворює домінування більш поширеного класу, а справді навчається відокремлювати прибуткові логістичні операції від збиткових.

Комплексна оцінка ефективності. На тестовій вибірці фінальна модель досягла Accuracy = 0.6746, Balanced Accuracy = 0.6177, ROC AUC = 0.6737, Average Precision = 0.7630, MCC = 0.2513, Cohen's Kappa = 0.2476 та Brier Score = 0.1999. Отримані результати свідчать про помірну, але практично корисну здатність моделі відокремлювати прибуткові замовлення від збиткових навіть в умовах дисбалансу класів.

Порівняння з baseline. Особливо показовими є метрики, стійкі до дисбалансу. Balanced Accuracy зросла з 0.5000 до 0.6177, Macro F1 — з 0.3941 до 0.6214, а MCC і Cohen's Kappa підвищилися з 0 до 0.2513 і 0.2476 відповідно. Це підтверджує, що модель має реальну дискримінаційну здатність, тоді як naive baseline фактично не вміє відрізнити один клас від іншого.

Аналіз за класами. Для збиткового класу модель показала precision 0.54, recall 0.43 і f1-score 0.48, а для прибуткового класу — precision 0.72, recall 0.81 і f1-score 0.76. Отже, модель краще розпізнає прибуткові замовлення, однак після оптимізації порогу демонструє також прийнятну здатність виявляти ризикові, збиткові операції.

Якість імовірностей. ROC AUC = 0.6737 проти 0.5000 у baseline означає, що модель виконує змістовне ранжування замовлень за ймовірністю прибутковості, а не видає однаковий бал для всіх об'єктів. Average Precision = 0.7630 також суттєво перевищує базове значення 0.6505, що свідчить про кращу якість ранжування прибуткових замовлень.

Калібрування прогнозів. Brier Score фінальної моделі дорівнює 0.1999, що є кращим за baseline-значення 0.2274. Це означає, що передбачені ймовірності моделі є точнішими та краще узгоджуються з фактичними результатами. Даний висновок підтверджується також кривою калібрування (рисунок 1).

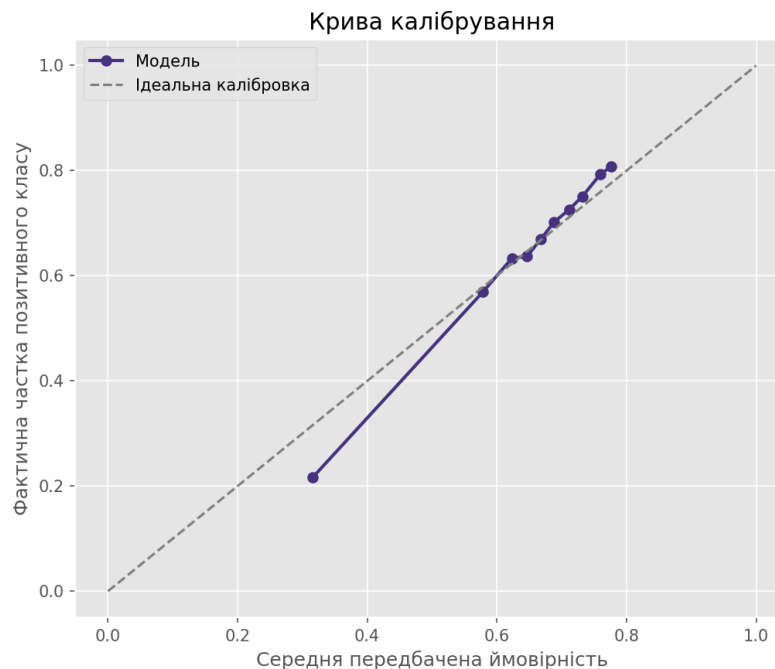


Рисунок 1 – Крива калібрування моделі

3.5 Аналіз ROC та Precision-Recall

На рисунку 2 показана ROC-крива. AUC = 0.6736 означає: якщо вибрати навмання одне прибуткове і одне збиткове замовлення, модель з високою ймовірністю дасть вищий бал саме прибутковому. Запропонований підхід зі збагаченням вектора ознак зовнішніми даними забезпечує стабільний і практично корисний результат для задачі відокремлення прибуткових операцій від збиткових.

Крива Precision-Recall (рисунок 3) показує Average Precision = 0.7628

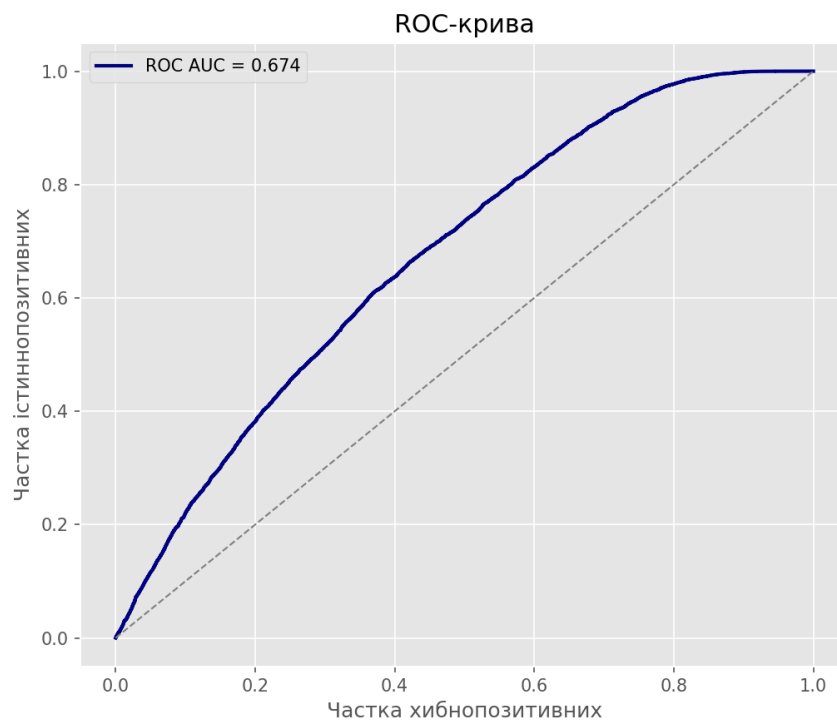


Рисунок 2 – ROC-крива моделі ($AUC = 0.6736$)

при базовому рівні 0.657 — це означає, що модель ранжує прибуткові за-
мовлення значно вище, ніж можна очікувати від випадкового розподілу.

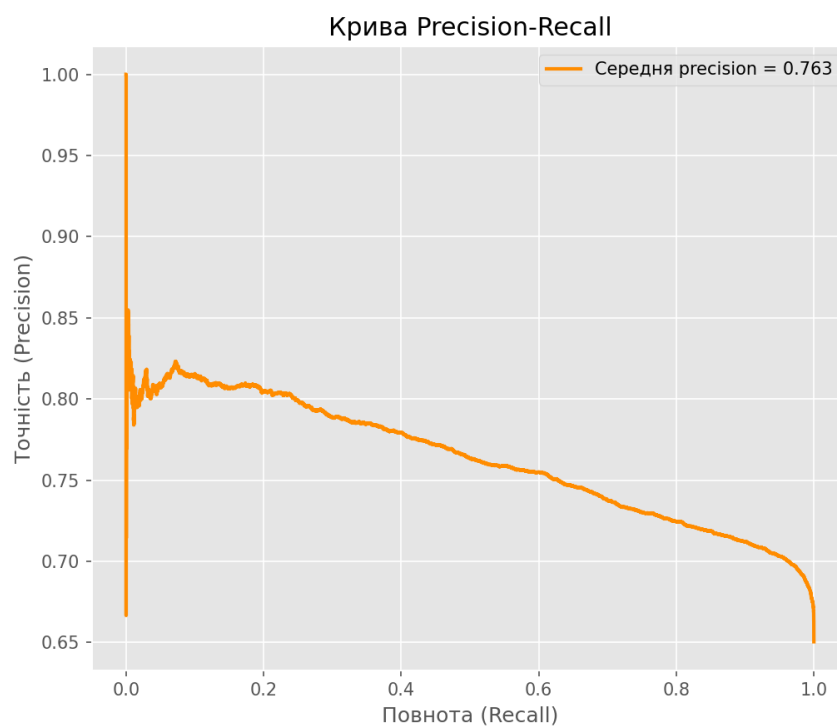


Рисунок 3 – Крива Precision-Recall ($AP = 0.7628$)

3.6 Оптимізація порогу класифікації

За замовчуванням поріг дорівнює 0.5, але це не завжди оптимально. Було проведено пошук порогу $\tau \in [0.05, 0.95]$ з кроком 0.01 на **Validation set**, максимізуючи Macro F1. Результат показано на рисунку 4.

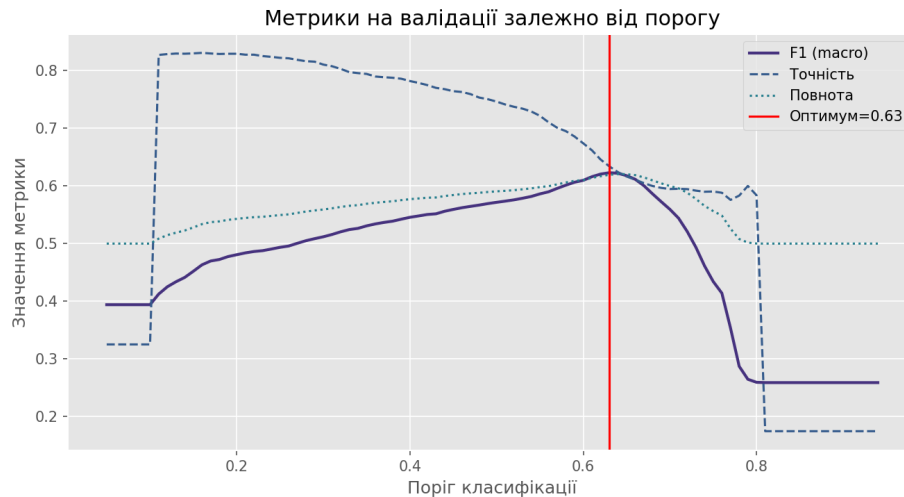


Рисунок 4 – Залежність Macro F1 від порогу класифікації

Для глибшого розуміння того, чому стандартний поріг 0.5 є неефективним для поточної задачі, доцільно проаналізувати розподіл передбачених ймовірностей для обох класів (рисунки 5).

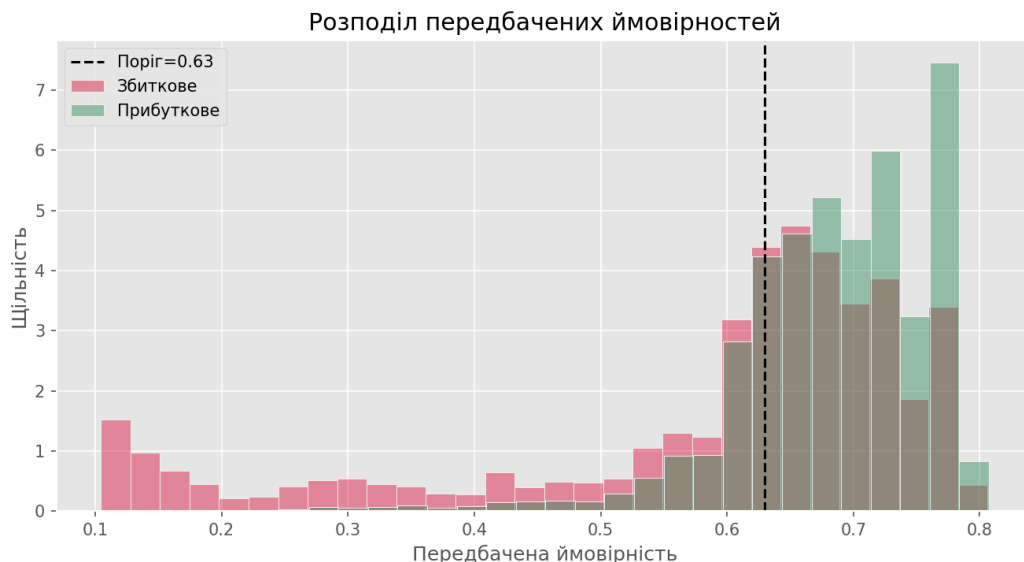


Рисунок 5 – Розподіл передбачених моделлю ймовірностей для прибуткових та збиткових замовлень

Як видно з гістограми, класи мають значне перекриття у діапазоні ймовірностей від 0.5 до 0.7. Багато фактично збиткових замовлень (рожеві

стовпці) отримують від моделі бал вище 0.5, що при стандартному порозі призводило б до їх класифікації як «прибуткових» (False Positives). Оптимальний поріг $\tau = 0.63$ (позначений чорною пунктирною лінією) проходить саме через ту точку, де щільність прибуткових замовлень починає суттєво домінувати над збитковими.

Оптимальним виявився поріг $\tau = 0.63$. При ньому Macro F1 = 0.6229 на Validation. Підвищення порогу до цієї позначки дозволяє утримувати Recall збиткового класу на рівні близько 0.43, тобто модель виявляє суттєву частину ризикових замовлень, не зводячи задачу до надмірно консервативних відмов. Ціна цього — деяка кількість прибуткових замовлень, яким відмовлено. Але в логістиці прийняти збиткове замовлення значно дорожче, ніж відмовитись від прибуткового, тому такий компроміс виправданий.

Результат при $\tau = 0.63$ на Test наведено у матриці помилок (рисунок 6).



Рисунок 6 – Матриця помилок при порозі 0.63

3.7 Інтерпретація через метод SHAP

Одна з проблем градієнтного бустингу — важко пояснити, *чому* модель прийняла конкретне рішення. Для вирішення цього було використано **SHAP** [2] — метод, що базується на теорії ігор (значення Шеплі). Основна ідея: для кожного замовлення і кожної ознаки SHAP рахує, на скільки ця

ознака «зрушила» прогноз моделі відносно середнього.

Насамперед варто оцінити загальну (глобальну) важливість сконструйованих ознак за допомогою аналізу середнього модуля SHAP-значень (рисунок 7). Цей показник демонструє абсолютний масштаб впливу кожної ознаки на фінальний прогноз моделі незалежно від напрямку цього впливу.

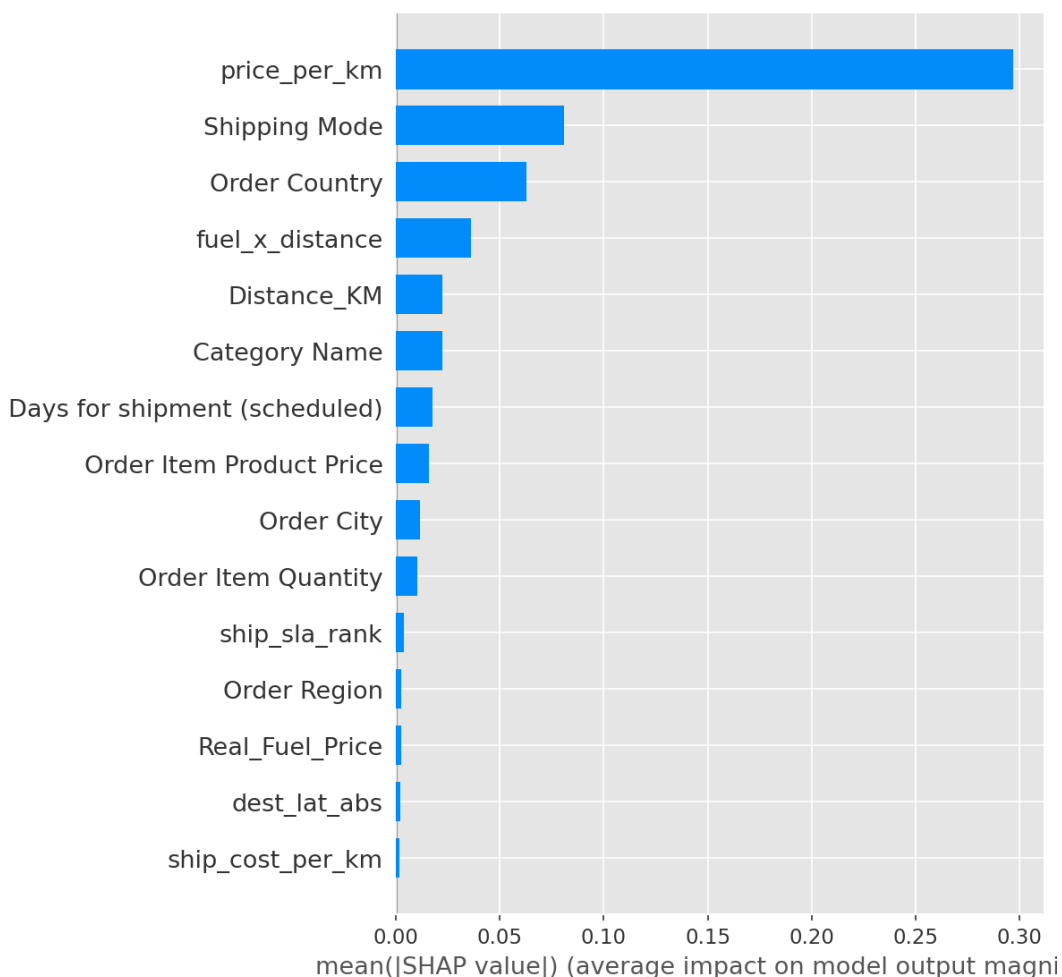


Рисунок 7 – Глобальна важливість ознак за середнім абсолютним SHAP-значенням

Як видно з графіка, беззаперечним лідером є сконструйована ознака `price_per_km`. Серед найбільш важливих також присутні ознаки, пов'язані з паливним навантаженням, режимом доставки та просторово-геоекономічним контекстом маршруту. Такий розподіл важливості підтверджує ефективність обраної методології: для прогнозування ключову роль відіграють саме інженерні ознаки, побудовані на основі логістичного змісту задачі.

Цю ж тенденцію підтверджує і стандартна важливість ознак за внутрішніми критеріями алгоритму XGBoost (рисунок 8), заснована на критерію Gain. Ознака `price_per_km` очікувано стала лідером і тут.

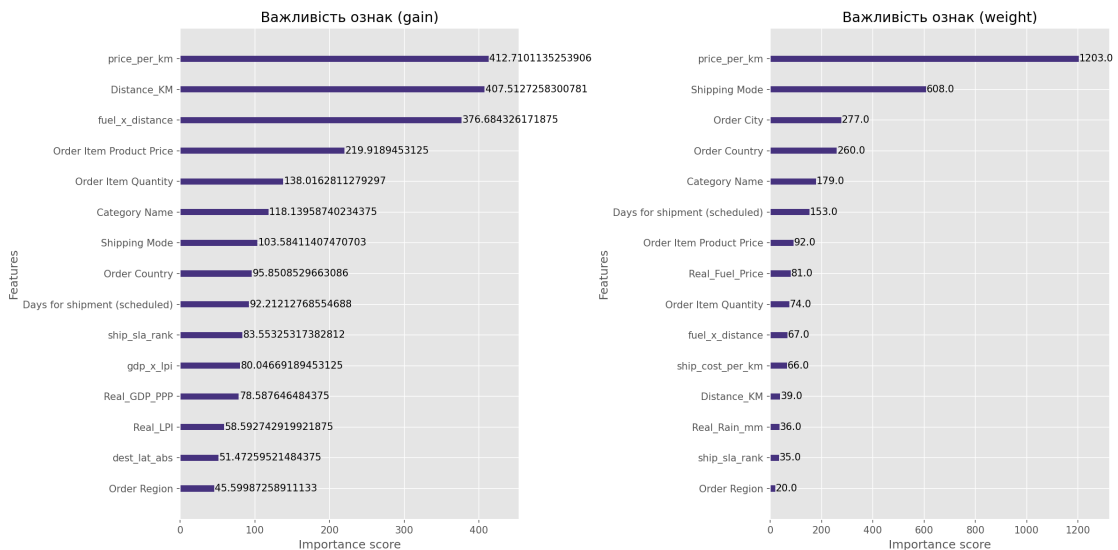


Рисунок 8 – Важливість ознак за критерієм Gain

Ще цікавіший графік Beeswarm (рисунок 9). Тут кожна точка — це одне замовлення. Положення точки по горизонталі показує, наскільки ця ознака змістила прогноз убік «прибутково» (вправо) або «збитково» (вліво). Колір показує значення ознаки: рожевий — висока, синій — низька.

Ключові висновки з SHAP:

- **price_per_km** — висока ціна замовлення на одиницю відстані (рожеві точки) стабільно штовхає прогноз у бік прибутку.
- **fuel_x_distance** — великі паливні витрати (далекі маршрути при дорогій нафті) тягнуть прогноз убік збитків.
- **Order Country** — країна призначення має великий вплив: це агрегований сигнал про митні ставки, LPI та GDP ринку.

Цей аналіз дозволяє пояснити логіку моделі простою мовою, що критично для впровадження в реальній компанії: менеджер хоче розуміти, *чому* система радить відмовитися від замовлення.

3.8 Приклад роботи системи

На завершення — конкретний приклад з CLI-режиму: маршрут зі складу в Нью-Йорку до Парижа, Standard Class, Electronics, 1 одиниця, \$500, знижка 10%.

Система розрахувала:

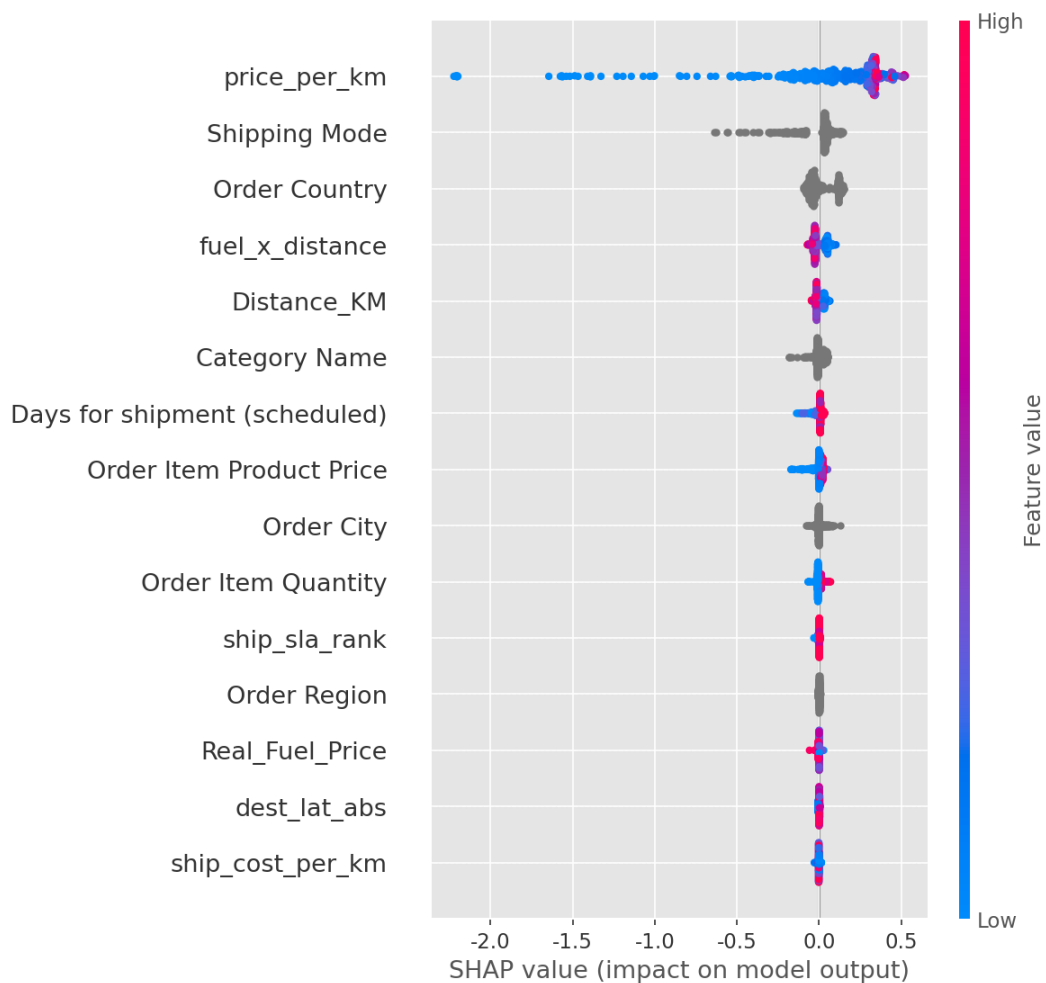


Рисунок 9 – SHAP Beeswarm-графік: напрям впливу ознак

- Відстань: 5 834.74 км (за формулою гаверсинусів);
- Паливний штраф: \$7.97 (відстань $\times$ Brent/60 $\times$ коефіцієнт Standard);
- Митний збір: \$0.00 (Electronics з США до Франції, нульова ставка);
- Погодний штраф: \$0.00 (опаді 1.4 мм, що нижче порогового значення для штрафу);
- Вердикт: **ПРИБУТКОВО**, ймовірність 73.40% (поріг 64.0%).

Результат логічний: товар має достатньо високу ціну відносно відстані ($\approx$ \$0.08/км — високе значення `price_per_km`), митний збір відсутній, а погодні умови не створюють додаткового штрафного навантаження.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено та досліджено систему класифікації логістичних операцій на прибуткові і збиткові на основі алгоритму XGBoost із використанням зовнішніх погодних, паливних і макроекономічних характеристик. На основі проведеної роботи можна сформулювати такі результати:

1. **Аналіз літератури та властивостей алгоритмів** показав, що для структурованих табличних даних із числовими та категоріальними ознаками алгоритми градієнтного бустингу є доцільним вибором. Саме тому в межах даної роботи основним алгоритмом реалізації та дослідження було обрано XGBoost.
2. **Усунуто проблему іспаномовних назв країн у DataCo:** розроблений чотирирівневий алгоритм нормалізації суттєво покращив якість прив'язки замовлень до макроекономічних характеристик країн і зменшив кількість помилкових значень за замовчуванням.
3. **Спроектовано формулу реалістичної прибутковості** з динамічними штрафами за відстань, погодні умови та митні збори з LPI-корекцією. Це дозволило перейти від формального показника прибутку в наборі даних до більш змістовної оцінки операційної ефективності замовлення.
4. **Покращено пайплайн формування ознак:** після впровадження багаторівневого fallback-механізму для координат і погодних даних вдалося усунути масове використання жорстких дефолтів. У фінальному датасеті використання фіксованої відстані 5000 км зведено до мінімуму, дефолтна температура 15.0°C повністю усунута, а кількість випадків використання нульових опадів як значення за замовчуванням стала поодиноким.
5. **Реалізовано тришаровий поділ Train/Validation/Test.** Validation-вибірка використовувалася для добору гіперпараметрів та оптимального порогу класифікації, а Test — для підсумкової оцінки якості моделі.

6. **Досягнуто покращених результатів моделі:** на тестовій вибірці фінальна модель показала $\text{ROC AUC} = 0.6737$, $\text{Accuracy} = 0.6746$, $\text{Balanced Accuracy} = 0.6177$, $\text{Average Precision} = 0.7630$, $\text{MCC} = 0.2513$, $\text{Cohen's Kappa} = 0.2476$ та $\text{Brier Score} = 0.1999$. Крос-валідація також підтвердила стабільність моделі: середнє значення ROC AUC склало 0.6573 при стандартному відхиленні 0.0025 .
7. **Оптимізація порогу класифікації** показала, що найкращим є поріг $\tau = 0.63$. При ньому Macro F1 на validation досягло 0.6229 , а recall збиткового класу на тестовій вибірці утримується на рівні близько 0.43 , що є практично важливим для виявлення ризикових замовлень.
8. **SHAP-аналіз** підтвердив, що найважливішим предиктором є ознака `price_per_km`, а серед найбільш впливових також присутні характеристики паливного навантаження, способу доставки та геоекономічного контексту маршруту. Це робить рішення моделі інтерпретованими та придатними для практичного використання.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

- [1] **Chen T., Guestrin C.** XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*. 2016. P. 785–794.
- [2] **Lundberg S. M., Lee S. I.** A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. 2017. P. 4765–4774. URL: <https://arxiv.org/abs/1705.07874>
- [3] **Akiba T., Sano S., Yanase T., Ohta T., Koyama M.** Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD*. 2019. P. 2623–2631.
- [4] **Pedregosa F. et al.** Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011. Vol. 12. P. 2825–2830.
- [5] **Chicco D., Jurman G.** The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*. 2020. Vol. 21. P. 6.
- [6] **Lesmana S. P., Hermawati D., Mukaromah M. та ін.** Machine Learning Implementation for E-commerce Delivery Delay Prediction Using XGBoost Algorithm. *Green Engineering: International Journal of Engineering and Applied Science*. 2025. Vol. 2, No. 3. P. 46–57.
- [7] **Sattar M. U., Dattana V., Hasan R. та ін.** Enhancing Supply Chain Management: A Comparative Study of Machine Learning Techniques with Cost–Accuracy and ESG-Based Evaluation for Forecasting and Risk Mitigation. *Sustainability* (MDPI). 2025. Vol. 17, No. 13. P. 5772.
- [8] **Nsir N., Alzubi A. B., Adegboye O. R.** Enhancing Sustainable Supply Chain Performance Prediction Using an Augmented Algorithm-Optimized XGBOOST in Industry 4.0 Contexts. *Sustainability* (MDPI). 2025. Vol. 17, No. 22. P. 10344. DOI: 10.3390/su172210344.
- [9] **Grinsztajn L., Oyallon E., Varoquaux G.** Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*. 2022. Vol. 35. P. 507–520.

- [10] **Sinnott R. W.** Virtues of the Haversine. *Sky and Telescope*. 1984. Vol. 68, No. 2. P. 158–159.
- [11] **Harris C. R. et al.** Array programming with NumPy. *Nature*. 2020. Vol. 585. P. 357–362.
- [12] **Open-Meteo**. Historical Weather Data Documentation. — Режим доступу: <https://open-meteo.com/en/docs> (дата звернення: 15.04.2026).
- [13] **Nominatim API**. OpenStreetMap Geocoding Service. — Режим доступу: <https://nominatim.org> (дата звернення: 15.04.2026).
- [14] **Constante F., Silva F., Pereira A.** DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS. *Mendeley Data*. 2019. V3. DOI: 10.17632/8gx2fvg2k6.3.
- [15] **World Bank**. Logistics Performance Index (LPI) Report. — Режим доступу: <https://lpi.worldbank.org> (дата звернення: 18.04.2026).
- [16] **GitHub репозиторій проєкту:**
<https://github.com/Delukich/Coursework.git> (дата звернення: 15.05.2026).